IT Helpdesk Ticketing System: Project Plan

Individual Student Project, Project 8
Student: [Antonie Potgieter] | Student number: [20260249]
Repository: [your GitHub link] | Stack: Python 3, Django, SQLite, Bootstrap 5

1. Project overview

An office logs IT problems by email and requests get lost. This system lets employees log tickets, an admin assign them to technicians, and technicians work each ticket through to resolution. Every status change and comment is kept as a history, and each ticket has a target resolution time based on its priority.

2. User roles

Role	What they can do
Employee	Register, log in, create tickets, view their own tickets, comment, close resolved tickets
Technician	View tickets assigned to them, update status, add comments
Admin	View all tickets, assign tickets to technicians, manage users and categories, view reports

3. User stories

Employee

- As an employee, I want to register and log in so that I can use the system securely.
- As an employee, I want to log a ticket with a title, description, category, priority and optional screenshot so that IT knows what is wrong.
- As an employee, I want to see the status of my tickets so that I know when my problem will be fixed.
- As an employee, I want to add comments to my ticket so that I can give more information.
- As an employee, I want to close a resolved ticket so that I can confirm the problem is fixed.

Technician

- As a technician, I want to see only the tickets assigned to me so that I know what to work on.
- As a technician, I want to update a ticket's status so that employees can follow progress.
- As a technician, I want to comment on a ticket so that I can ask questions or explain the fix.

Admin

- As an admin, I want to see all tickets so that nothing is lost.
- As an admin, I want to assign a ticket to a technician so that someone is responsible for it.
- As an admin, I want overdue tickets highlighted so that I can act on them quickly.
- As an admin, I want reports (open tickets per technician, average resolution time, tickets by category) so that I can manage the workload.

System

- The system stores passwords hashed and never in plain text.
- The system validates all form input on the server and shows clear error messages.
- The system records every status change and comment as a history on the ticket.
- The system calculates a due time from the ticket's priority (High 4 hours, Medium 24 hours, Low 72 hours).
- The system only allows valid status changes: Open to Assigned to In Progress to Waiting on User or Resolved, then Closed.

4. Feature list

Core scope

1. Register, log in and log out with hashed passwords
2. Three roles with different permissions
3. Create tickets (title, description, category, priority, optional screenshot)
4. Ticket statuses: Open, Assigned, In Progress, Waiting on User, Resolved, Closed
5. Admin assigns tickets to technicians
6. Technicians update status and add comments
7. Full history of every status change and comment

8. Target resolution times by priority, with overdue tickets highlighted
9. Reports: open tickets per technician, average resolution time, tickets by category
10. Search and filter tickets

Baseline requirements

- Relational database with at least 4 related tables, plus a seed script
- Server-side validation on every form
- At least 10 automated tests
- Responsive interface (laptop and phone)
- Runs from the README in under 10 minutes

Stretch goals (only if all core features work)

- Email notifications when a ticket is assigned or its status changes
- Automatic assignment by category
- Customer satisfaction rating after closing

5. Database design

The database has six tables. The ERD diagram is saved in the repository as docs/erd.png.

[docs/erd.png]

Table	Purpose	Key fields
User	Accounts and roles	id, username, email, password (hashed), role
Category	Ticket categories	id, name
SLAPolicy	Target hours per priority	id, priority, hours
Ticket	The IT problem	id, title, description, priority, status, screenshot, category, created_by, assigned_to, created_at, due_at, resolved_at
Comment	Discussion on a ticket	id, ticket, author, text, created_at
StatusHistory	Audit trail of status changes	id, ticket, changed_by, old_status, new_status, changed_at

Relationships:

- One User creates many Tickets, and one User (a technician) can be assigned many Tickets.
- One Category has many Tickets.
- One Ticket has many Comments and many StatusHistory records.
- SLAPolicy is a lookup table matched to a ticket by its priority, so it has no foreign key.

6. Wireframes of the main screens

The wireframes are saved in the repository as docs/wireframes.png.

[docs/IT Helpdesk wireframes.html]

Screen	Layout
Login	Centred card with email/username and password fields, a Log in button, and a Register link
Register	Centred card with username, email, password and confirm password
Ticket list	Navy top bar, search box and status filter, table of tickets with coloured status and priority pills, overdue rows in light red, and a New ticket button
New ticket	Single-column form: title, category, priority, description, screenshot upload, Submit and Cancel
Ticket detail	Ticket information and comments on the left; assign, update status, due time and history panels on the right
Reports (admin)	Four summary cards, then average resolution time, open tickets per technician and tickets by category

Design: dark navy navigation bar, indigo (#4F46E5) buttons, light grey page background, rounded white cards, and consistent colours for each status and priority.

7. Week-by-week plan

Week	Focus	Deliverables	Check-in
1	Plan and set up	Selection form, repository, Django project, project plan with ERD and wireframes, database models	Plan approved
2	Build the core	Registration, login and roles, seed script, create and list tickets	Login and database working
3	Complete features	Assignment, status workflow, comments and history, SLA deadlines, reports, validation, automated tests	Core features done
4	Test and present	Bug fixes, manual test table, README, technical report, demo video, final submission and presentation	Final submission

Commit to Git at least 3 times a week with clear messages.

8. Risks and how I will manage them

Risk	Plan
Running out of time	Finish all core features before any stretch goal
Setup or environment errors	Use a virtual environment and keep requirements.txt up to date
Role permissions have gaps	Check roles on the server in every view, and write tests for them
Losing work	Commit and push to the private GitHub repository regularly

9. Testing approach

- Automated tests with Django's test framework: SLA calculations, status transitions, role-based access, validation and password hashing (at least 10 tests).
- A manual test table with expected and actual results for each screen.
- Test logins for each role are created by the seed script and listed in the README.